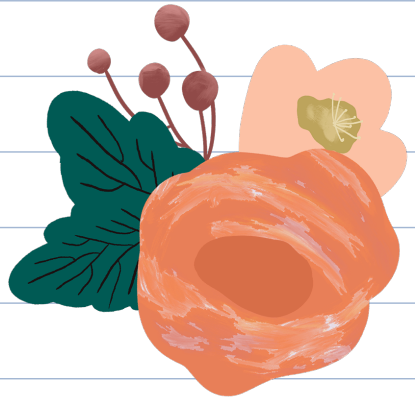

Good Evening Everyone :)

Starting at 9:02pm.

PAINTER'S PARTITION

We have to paint all N boards of length $[C_0, C_1, C_2, \ldots C_{N-1}]$
There are K painters available and each of them takes 1 units of time to paint 1 unit of board.

Calculate minimum time required to get job done.

NOTE :

- ✓ Two painters cannot share a board to paint.
- ✓ A painter can only paint continuous boards. This means a painter painter continuous subarray.

K=3

A[] = { 3 5 1 7 8 2 5 3 10 1 4 7 5 4 6 }

31 25 15

K=3

A[] = { 3 5 1 7 8 2 5 3 10 1 4 7 5 4 6 }

26 23 22

K=3

A[] = { 3 5 1 7 8 2 5 3 10 1 4 7 5 4 6 }

24 25 2

25

Example 2

$A = [10 \quad 20 \quad 30 \quad 40]$

$K = 2$

P1 P2 $\Rightarrow$ 70

P1 P2 $\Rightarrow$ 60

GREEDY APPROACH

Take total and divide by K,

A[] = [1 2 3 4, 100]. K = 2

Greedy X

Quiz 1 :

[1 2 3 4 100] $K=2$

A[] = { 2 , 5 , 3 , 8 } $K=4$

Binary Search Approach

lets look at example below:-

{ 3 5 1 7 8 2 5 3 10 1 4 7 5 4 6 } K = ~~4~~ ~~3~~ ~~2~~ 1

P1 P2 P3 P4

Search Space

Best case :- max of all elements.

Worst Case : Sum of all elements

Target : I need to find maximum time to complete the task

Painter's Partition: Dry Run & Pseudocode

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	5	1	7	8	2	5	3	10	1	4	7	5	4	6

$k = 4$

$lo = 10$ $\quad$ $hi = 71$

$$mid = 10 + \frac{(71-10)}{2} = 40$$

Check whether 4 painters will complete task in 40 minutes

$P1 = 3+5+1+7+8+2+5+3 = 34.$

$P2 = 10+1+4+7+5+4+6 = 37.$

$hi = mid - 1 = 39$

$lo = 10$ $\quad$ $hi = 39$ $\quad$ $mid = 24.$

$P1 = 3+5+1+7+8 = 24$

$P2 = 2+5+3+10+1 = 21$

$P3 = 4+7+5+4 = 20$

$P4 = 6$

$Hi = mid - 1 = 24 - 1 = 23$

$lo = 10$ $Hi = 23 = 16$

	2	3	4	5	6	7	8	9	10	11	12	13	14
{ 3	5	1	7	8	2	5	3	10	1	4	7	5	4 6]

$P1 = 3+5+1+7 = 16$

$lo = 16+1 = 17$

$lo = 17$ $Hi = 23$ $mid = 20$

	2	3	4	5	6	7	8	9	10	11	12	13	14
{ 3	5	1	7	8	2	5	3	10	1	4	7	5	4 6]

$lo = 21$ $Hi = 23$ $mid = 22$

	2	3	4	5	6	7	8	9	10	11	12	13	14
{ 3	5	1	7	8	2	5	3	10	1	4	7	5	4 6]

$Hi = mid - 1 = Hi = 21$

$lo = 21$ $Hi = 21 = mid = 21.$

PSEUDOCODE

```
paintersNumber (arr[], N, mid, K) {

    res = 0          , no. of painters = 1

    for (i = 0 to N-1) {

        res = res + arr[i]

        if (res > mid)
            res = arr[i]
            numofpainters ++
        }

    }

    if (numofPainters <= K) {

        return true
    }

    return false

}
```

```
partition (arr[], N, K) {

    lo =      maxEle (arr, N)
    Hi =      SumArr (arr, N)

    while (lo <= hi) {

        mid =     lo + (hi - lo)/2

        if (paintersNumber (arr, N, mid, K) {

            ans = mid
            hi = mid-1
        }
        else {

            lo = mid+1
        }
    }
    return ans

}
```

Email Handler

```
//Email handling

// task : design a email handling  solution ,
// you have x email handlers -> workers // handle email
// you have n emails to handle-> every email have a complexity atttached to i

[ 23. 5    34  12]  x=2


}
```

AGRESSIVE COWS

Given N cows and M stalls, all M stalls are located in different location of x-axis. place all the cows such that minimum distance b/w two cows is maximized

There can be only one cow in a stall at a time

We need to place all cows.

Testcase 1

stalls[] = [1 2 4 8 9]

cows = 3

stall = 5

					Min distan	
C1	C2	C3			(1, 3, 2)	1 ⇒
C1		C2		C3	(3, 5, 8)	3
C1			C2	C3	(7, 1, 8)	1

Testcase 2

stalls[] = [2 6 11 14 19 25 30 39 43]

cows = 4

C1	C2	C3	C4	⇒ 3.				
C1		C2		C3		C4	= 8	
C1			C2			C3	C4 = 12	

Quiz 3

Quiz 4.

$A = [\,0 \quad 3 \quad 4 \quad 7 \quad 9 \quad 10\,]$

$K=4$

C1 C2 C3 C1

3 4 3

$\Rightarrow$ 3.

Aggressive Cows Binary Search

let's use testcase 2 to get the intuition of the question

Search Space

A[] = { 4 7 14 20 } K = 4

↳ Ans = 3.

Worst Case

cows = n.

Best Case

distance = A[n-1] - A[0].

There are only two cows, we place them at first and last.

S — Space [1 A[N-1] - A[0]].

Condition → if mid distance is maintained, then look for better distance at right.

Example

If 20 can be maintained :

$k=4$

A[] = 2 6 11 14 19 25 30 39 43

C1 C2

Only 2 cows are placed ✗ doesn't work.

A[] = 2 6 11 14 19 25 30 39 43

$K=4$.

C1 C2 C3 C4 min dis $= 5$.

✓ 5 can be maintained.

```
// function to check if we are able to maintain a distanc of x
// between c cows

1  4 7 10 23 x=7
c1     c2 c3
function check(ar[] , x, c)
{
    n = ar.length
    int count =1
    last =0 //last placed cows
    for( i> 1 to n-1)
    {
        if(ar[i] - ar[last] >=x)
        {
            last =i // cow is placed
            count ++
        }
    }
    if(count >= c)
    {
        return true
    }
    return false
}
```

```
   Tc: O(N* log(mindistance- distance b/w extreme end)
// function to find the minimum distanve which can be maintained
aggresive_cows(ar [], size , cows){

    lo=0;
    ans=-1
    for( i->1 to n-1)
    {
        lo = min(lo , ar[i]-ar[i-1])
    }
   // 1 3 5 7 9 k=5 lo =2

   hi= ar[n-1]- ar[0]


   //Apply Binary Search

   while(lo<=hi){
       mid= lo+ ((hi-lo)/2)
       if(check(ar ,mid , cows)){
           ans = mid //atleast mid distance is maintained
           lo= mid+1 // better ans at right
       }
       else{
           hi= mid-1
       }
   }
```

HINTS | Hint 1 | Solution Approach | Complete Solution

Editor Mode - Normal | Python 3 (python-3.8)

```python
class Solution:
    # @param A : list of strings
    # @return a list of strings
    def solve(self, A):
        A.sort(key =lambda x: len(x), reverse= True)
        return A
```

```java
public class Solution {
    public ArrayList<String> solve(ArrayList<String> A) {
        Collections.sort( A, new Comparator<String>(){

            public int compare(String a, String b)
            {
                int len1 = a.length();
                int len2 = b.length();
                //return Integer.compare(len1 ,len2)

                if(len2 >len1){
                    return 1;
                }
                else if( len1 > len2)
                {
                    return -1;
                }
                return 0;
            }
        });
        return A;
    }


}
```